Listing 9.3. Sort competitors from stream

```
fs = require 'fs'
{EventEmitter} = require 'events'

ONE_SECOND = 1000

lastName = (s) ->
  try
    s.split(/\s+/g)[1].replace /,/, ','
  catch e
    ''

undecorate = (array) ->
  item.original for item in array
class CompetitorsEmitter extends EventEmitter

  validCompetitor = (string) ->
    /^[0-9]+:\s[a-zA-Z],\s[a-zA-Z]\n/.test string

  lines = (data) ->
    chunk = data.split /\n/
    first = chunk[0]
    last = chunk[chunk.length-1]
    {chunk, first, last}


  insertionSort = (array, items) ->
    insertAt = 0
    for item in items
      toInsert = original: item, sortOn: lastName(item)
      for existing in array
        if toInsert.lastName > existing.lastName
          insertAt++
      array.splice insertAt, 0, toInsert

  constructor: (source) ->
    @competitors = []
    stream = fs.createReadStream source, {flags: 'r', encoding: 'utf-8'}
    stream.on 'data', (data) =>
      {chunk, first, last} = lines data
      if not validCompetitor last
        @remainder = last
        chunk.pop()
      if not validCompetitor first
        chunk[0] = @remainder + first
      insertionSort @competitors, chunk
      @emit 'data', @competitors

path = require 'path'
if !fs.existsSync 'competitors.15000.txt'
  console.error 'Error: File competitors.15000.txt not found'
  process.exit()

competitors = new CompetitorsEmitter 'competitors.15000.txt'
competitors.on 'data', (competitors) ->
    console.log "There are #{competitors.length} competitors"

start = new Date()
setInterval ->
  now = new Date()
  console.log "Tick at #{(now - start)/ONE_SECOND}"
, ONE_SECOND/10
```

Check that competitor names have the expected format.

Read the lines from the data provided.

Sort the competitors by inserting them in the correct order as they are received.

Read the file as a stream and then read the line from each chunk of the stream received.